Hidden in the Leftovers

Task-Specific Reconstruction Gaps of Sparse Autoencoders
in English and Indian Languages

Prakhar Singhal (2022111025, prakhar.singhal@research.iiit.ac.in)
Raveesh Vyas (2022114002, raveesh.vyas@research.iiit.ac.in)

Mechanistic Interpretability of Language Models (CL3.412)
Course project proposal · Team IpCp-- · October 2026

Summary. Sparse autoencoders (SAEs) are widely used to interpret language-model activations and are usually judged by average reconstruction quality, on which the Gemma Scope SAEs score 98–99%. In a pilot study on gemma-2-2b we exchange parts of the activations between two prompts (activation patching) to measure how much of the information the model uses for a task the SAE's features carry. For object tracking at layer 12 they carry 74%; one direction of the SAE error, estimated on that task, recovers about half of the missing 26%, while directions from other tasks or generic text recover almost none. We propose to test how general such task-specific gaps are, whether they reflect binding, and whether they are larger in Hindi and Hinglish.

1 What: research question and hypothesis

A language model reads text as **tokens** (words or word pieces) through a stack of **layers**; at each layer it holds an **activation** x (2304 numbers per token in gemma-2-2b). A **sparse autoencoder** (SAE) writes $x = \hat{x} + \varepsilon$: the **features** $\hat{x}$ are a reconstruction from a few interpretable components, and the **SAE error** ε is the part it does not explain ("dark matter" [4]; informally, the leftovers). A **direction** is a fixed vector in the activation space. Appendix E defines all terms.

Research question. *When an SAE's features fail to carry information that a model uses for a task, is the missing part specific to that task, and is more of it missed in Indian languages?*

Hypothesis. We call the share of a task's effect that the features fail to carry the SAE's **gap** on that task. We hypothesise that gaps are of two kinds: a *concentrated, task-specific* gap is largely closed by one or two error directions estimated on the task itself, a *diffuse* gap needs many. Average reconstruction quality detects neither; the pilot (Section 3) found both. We ask:

- **Q1. Generality.** Which kinds of task show a concentrated, task-specific gap, and is the gap tied to one prompt template or to a type of computation?
- **Q2. Mechanism.** In object tracking, is the error direction part of the model's *binding* mechanism, the way it links an object to its location [5]?
- **Q3. Language.** Are the gaps larger in Hindi and Hinglish than in English, and do the same directions close them?

2 Why: motivation

SAE suites such as Gemma Scope [9] are used to explain, audit and steer models, and are evaluated mainly by **reconstruction fidelity**, the fraction of next-token performance kept when $\hat{x}$ replaces x: 98–99% for the 16k SAEs at layers 10–18 in our measurement. Fidelity is an average and cannot show what is lost on a particular computation, while the SAE error is known to be large, partly predictable from the input [4], and important inside feature circuits [11]. If what SAEs miss were generic, one remedy (a wider SAE, or a correction fitted on generic text) would serve all tasks; if it is task-specific, a feature-based explanation of a task can be incomplete in a way fidelity does not reveal. The same question arises across languages: the Gemma Scope SAEs were trained on predominantly English text [9], and prior work identifies language-specific SAE features [1, 2] but does not measure what the error carries in each language.

3 What has been done: pilot study

Setup. gemma-2-2b, an open 2-billion-parameter model [6], with Gemma Scope SAEs of 16k (16,384) to 262k features (Appendix B.1). Each task comes as pairs of prompts that differ in one step, so their

Table 1: The four pilot tasks, with one real pair each. Prompt B changes one step of prompt A.

Task	Prompt A (answer)	Change in prompt B (answer)
Running sum	`Start with 2. Add 3. Add 1. Add 2. Total =` (8)	`Add 1` becomes `Add 3` (10)
Object tracking [8]	`The pen is in box B. The coin is in box G. The key is in box C. The coin moves to box A. The pen moves to box G. The coin is in box` (A)	the second move is made by the coin (G)
Variable copying	`y = 4. r = 4. x = r. r = 3. x =` (4)	`r = 4` becomes `r = 7` (7)
Names (IOI) [14]	`When Lucy and David went to the river, David gave a drink to` (Lucy)	Lucy is the giver (David)

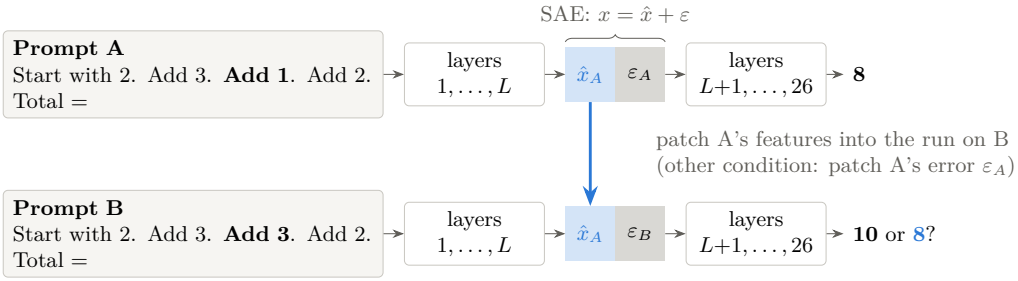


Figure 1: Activation patching with an SAE. The SAE splits each activation at layer L into features $\hat{x}$ and error ε. Patching A's features into the run on B and checking whether the answer moves from 10 to 8 shows how much of the task's information the features carry; the second condition patches A's error instead.

answers differ (Table 1); 300 pairs are used for fitting and 150 unseen pairs for every reported number. We first planned three other tasks but dropped them because the model did no better than a simple shortcut on them, such as repeating the starting number (Appendix A).

Method: activation patching [15] (Figure 1). We run the model on prompt B and, at one layer, replace one part of its activation ($\hat{x}$ or ε) with the same part from prompt A, at every token from the first one where A and B differ. The **effect recovered** is how far this moves the model's preference from B's answer towards A's (0: no change; 1: as far as replacing the whole activation). The gap is one minus the effect recovered by patching the features alone.

Findings (Appendix C):

1. **Features carry most of the effect.** On the three tracking tasks, patching $\hat{x}$ recovers 74–100% at every layer tested (4 to 20); patching ε recovers 0–13%, except object tracking at layers 8–16 (25–62%), which is mostly an artefact: replacing B's error with noise of the same size, which merely erases it, already recovers 28–58%. The names control loses 24–36% at layers 18–20.
2. **A concentrated gap.** For object tracking at layer 12, $\hat{x}$ recovers 74% (95% CI 73–76%). Also restoring A's error along the direction in which the task's training errors vary most raises this to 88%, about half of the gap; a random direction adds nothing (Figure 2).
3. **Task specificity.** Two same-task directions close 39–53% of that gap; directions from other tasks close −1 to 2%, from generic text 1%. The running sum at layer 18 (gap 13%) behaves alike (25–58% vs at most 1%); the names gap at layer 18 is diffuse (about 32 directions to close half).
4. **Width.** From 16k to 262k features the object-tracking gap goes 26, 20, 24, 23, 15%, and one error direction still closes 51–69% of it; wider SAEs also use more active features, so width stays open.
5. **Pitfalls.** Patching only the last token, probing without a prompt-only baseline, and equating what a probe can read with what the model uses would each have misled us (Appendix C.5).

Limitations. One model, one SAE family, synthetic tasks, width at one layer; patching along a few directions can create effects the model does not normally use [10], which our controls address only in part.

4 How: plan and success criteria

Q1, generality. We group tasks by the computation they need: *entity binding* (objects in boxes; people and their cities, e.g. "Asha lives in Pune. Ravi lives in Delhi. Asha lives in"), *counting* (longer running sums), *reference following* (variable copying) and the names control. Per task, layer and width we measure the gap, the directions needed to close half of it, and a **specificity score** (share

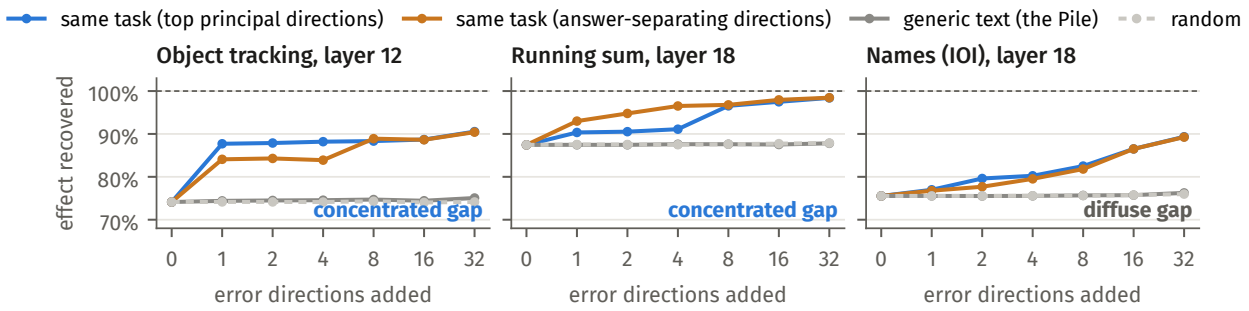


Figure 2: Concentrated and diffuse gaps. Effect recovered when the features are combined with A's error along N directions. For object tracking and the running sum, one or two directions estimated on the same task close much of the gap; directions from generic text or random directions close nothing. The names gap closes only slowly.

closed by same-task directions minus the largest share from any other source). Transfer between the two binding tasks separates a template-specific gap from a computation-specific one; SAEs with equal numbers of active features separate width from sparsity.

Q2, mechanism. Feng and Steinhardt [5] found that LLaMA and Pythia models mark each entity and its attribute with a shared "binding ID" vector. We will estimate these for gemma-2-2b, compare them with the object-tracking error direction, and shift activations along it to test whether the answer moves to the other box.

Q3, language. The same tasks in Hindi (Devanagari), Hinglish (Latin script, with English words), Hindi in Latin script without English words, Tamil or Bengali where the model can do them, and French as a non-Indian control. Example (draft, to be checked by a native speaker): शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = → 8 (design in Appendix D).

Table 2: Success criteria, fixed in advance. All comparisons use 95% confidence intervals; a result that meets neither column is reported as inconclusive.

Question	Supported if	Not supported if
Q1	gap $\geq$ 10%, and same-task directions close $\geq$ 30 points more than any other source; transfer between binding tasks $\geq$ half of own	on $\geq$ 2 new tasks with a gap, other sources close as much as same-task directions
Q2	shifting along the direction moves $\geq$ 50% of pairs to the other box; a random direction $\leq$ 5%	the shift is no more effective than a random direction
Q3	Hindi gap larger than English, with non-overlapping intervals (fidelity and transfer criteria in Table D.1)	Hindi gap not larger

Deliverables: (1) a table of gap size, type and specificity for every task, layer, width and language; (2) the patching code, with the three pitfalls handled; (3) a result on the binding hypothesis. **Resources:** one 8 GB GPU and the tested pilot pipeline (the main pilot runs took about 6.3 hours); risks and fallbacks are listed in Appendix D.

5 When: timeline

Table 3: Timeline. Each half-week ends with a checkpoint.

Dates	Work	Checkpoint
Oct 19–26	Pilot study, four stages (done; Appendix B.6)	concentrated gap found
Oct 27–29	Q2: binding vectors and the shifting test; Q1: new tasks	Q2 result
Oct 30–Nov 2	Q1: matched-sparsity widths; Q3: task performance in Hindi, Hinglish	usable languages
Nov 3–9	Q1: gap and specificity per task; Q3: fidelity, gap, transfer	Q1 and Q3 tables
Nov 10–12	Buffer; more test pairs; robustness checks	final figures
Nov 13–16	Report and final presentation	submission

References

[1] J. Brinkmann et al. Large language models share representations of latent grammatical concepts across typologically diverse languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6131–6150, Apr. 2025. Association for Computational Linguistics.

[2] B. Deng et al. Unveiling language-specific features in large language models via sparse autoencoders. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4563–4608, July 2025. Association for Computational Linguistics.

[3] S. Doddapaneni et al. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426, July 2023. Association for Computational Linguistics.

[4] J. Engels, L. R. Smith, and M. Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025.

[5] J. Feng and J. Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[6] Gemma Team et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[7] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Nov. 2019. Association for Computational Linguistics.

[8] N. Kim and S. Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, July 2023. Association for Computational Linguistics.

[9] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Nov. 2024. Association for Computational Linguistics.

[10] A. Makelov et al. Is this the subspace you are looking for? An interpretability illusion for subspace activation patching. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Marks et al. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[12] G. Penedo et al. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Second Conference on Language Modeling (COLM)*, 2025.

[13] S. Rajamanoharan et al. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

[14] K. R. Wang et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[15] F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Appendices

The original plan, the pilot methods and every pilot result

A From the original plan to the pilot tasks
B Pilot methods: model and SAEs, tasks, patching, repair, probes, runs
C Pilot results: patching, repair, specificity, width, pitfalls, replication, verdict, caveats
D Design of the planned study
E Glossary

A From the original plan to the pilot tasks

A.1 Original hypotheses

The project began as *PRISM* (Probing Residual Information and State Manifolds), a test of a strong version of the "dark matter" hypothesis: that SAEs fail to represent the step-by-step state a model computes, so that this state resides in the SAE error. It made three hypotheses, each tied to one experiment (Table A.1).

Table A.1: The original hypotheses and the experiment designed to test each.

	Hypothesis	Experiment	Prediction if true
H1	The SAE error carries the task state	Patch $\hat{x}$ or ε from prompt A into the run on prompt B	patching ε recovers more than patching $\hat{x}$
H2	The state forms a low-dimensional structure in the error	Spread of ε across directions; probes for the state; low-dimensional projections	a probe reads the state from ε better than from $\hat{x}$ and better than from the prompt's words
H3	A few error directions repair the SAE	Features plus A's error along its top r directions	a small r closes most of the gap; random directions do not

A.2 Original tasks and planned runs

Three algorithmic tasks whose answer depends on a hidden running state: a sum with a remainder (modular arithmetic), the depth of nested brackets, and the size of the graph reachable from a node, with the names task as a control. The planned runs used layers 4–24 (every fourth), SAE widths 16k and 65k, 400 training and 200 test pairs, up to 64 error directions and two extra direction types (top directions of $\hat{x}$; directions spanning both parts). They were organised in tiers for a single 24 GB GPU: a core tier (six layers × two widths × two patch positions), a tier over all 26 layers, a width tier (16k to 524k) and an expansion tier (1000/500 pairs, a second seed, generic-text directions).

A.3 The overnight run of October 19 and the code checks

The overnight run kept no usable pairs: after filtering, the three algorithmic tasks retained 10%, 21% and 16% of candidate pairs, the names control was dropped as unsolvable under that run's prompt format, and the core tier produced no results. A code check on October 20 then found pipeline problems as well as model limits: the few-shot examples were drawn after the training and test pairs had used up the prompt space (0 of 2 examples were added); the bracket-depth and names tasks could not fill their test sets (0 of 1400 and 144 of 600 candidates); the zero-shot names prompt was answered correctly only 69% of the time; and the installed transformers library (5.x) is untested with TransformerLens. All were fixed before the pilot: examples are now drawn first, and each task's prompt format, number of examples and difficulty were chosen in the capability study below. The same check confirmed that our model matches Hugging Face's implementation: the same top answer on 95% of prompts (largest logit difference 4.7, median 0.49 in 16-bit precision), correct answers on 6 of 7 sanity prompts for both (both miss "The capital of France is"), identical outputs when a run resumes from a middle layer, and identical scores in batched and single runs.

A.4 Capability study

On October 20 we measured each task on 128 pairs per setting, with 0, 2, 4 or 8 worked examples and several difficulty levels. A setting is usable if the model's top answer is correct on at least 60% of

prompts, at least 30% of pairs pass the pair filter, and the lower end of the 95% (Wilson) interval beats the best shortcut, such as repeating the starting number or the last number added. The three original tasks failed at every setting (Table A.2); for bracket depth, 83% of wrong answers simply repeated the starting depth. A probe could read bracket depth from the activations (99%), but a probe on the prompt's tokens alone did as well (100%), so this was not evidence of internal state.

Table A.2: Capability study. Original tasks at their planned difficulty; replacement tasks at the setting used in the pilot runs ("kept in run" is the share of generated pairs that passed the pilot's pair filter).

Task	Setting	Top answer correct	Best shortcut	Kept in run
Sum with remainder	4 steps, best of 0–8 examples	13%	21%	–
Bracket depth	8 operations, best of 0–8 examples	41%	41%	–
Graph size	4 edges, best of 0–8 examples	30%	48%	–
Running sum	3 steps, 8 examples	98%	26%	62%
Object tracking	2 moves, 8 examples	98%	65%	27%
Variable copying	1 hop, 2 examples	22%	16%	40%
Names (IOI)	2 examples	100%	17%	100%

Variable copying is weak at 2 examples in the capability study (it reaches 95% with 8), but the pilot runs used 2 examples with a stricter filter and a larger pool of candidates, which still left 300 training and 150 test pairs of cases the model solves.

B Pilot methods

B.1 Model and SAEs

Table B.1: Model, SAEs and their quality.

Item	Value
Model	gemma-2-2b: 26 layers, 2304 numbers per token, run in 16-bit (bf16) precision
SAEs	Gemma Scope residual-stream SAEs with a JumpReLU activation [9, 13] (`gemma-scope-2b-pt-res-canonical`); widths 16k, 32k, 65k, 131k and 262k
Fidelity (16k)	next-token performance kept on generic text: 99.1% (layer 10), 98.7% (12), 98.5% (14), 98.0% (18), relative to replacing the activation by its average; 3936 tokens of the Pile
Error size (16k)	$\|\varepsilon\|/\|x\|$ on the pilot prompts: 0.08–0.09 at layer 4, 0.12–0.18 at layers 8–16, 0.11–0.13 at layer 20
Compute	one 8 GB laptop GPU (RTX 5060)

B.2 Tasks

Each task generator produces pairs whose prompts have the same number of tokens and whose answers are single tokens; pairs that break this are dropped and counted. Worked examples are prepended identically to both prompts. The **pair filter** keeps only pairs the model handles: *strict* requires the model's top answer to be correct on both prompts and its preference for A's answer over B's to exceed 2 logits on prompt A; *clean-only* requires a correct top answer on prompt A and a change in preference of more than 2 logits between the prompts. A task is skipped if fewer than 20 pairs survive.

Table B.2: Task settings in the main pilot. "Preference" is the mean logit difference between A's and B's answer on prompt A and on prompt B; chance is one over the number of possible answers.

Task	Difficulty	Examples	Filter	Kept	Preference A / B	Chance
Running sum	3 steps	8	strict	62%	3.48 / −3.02	25%
Object tracking	3 objects, 2 moves	8	clean-only	27%	2.77 / −0.15	18%
Variable copying	1 copy hop	2	strict	40%	3.11 / −2.81	11%
Names (IOI)	–	2	strict	100%	6.75 / −6.39	11%

The running sum also uses a story form ("Anna has 5 cards. Anna gets 1 more. Anna gets 2 more.

Anna gets 1 more. Anna now has" → 9; in B the second step becomes "gets 1 more" → 8). Variable copying always opens with a distracting assignment, so that "the first number" is not a shortcut.

B.3 Activation patching

Effect recovered (NLDR, normalized logit-difference recovery). A *logit* is the model's raw score for a possible next token. With D the logit of A's answer minus the logit of B's answer, NLDR $= (D_{\text{patched}} - D_B)/(D_A - D_B)$, averaged over test pairs: 0 means no change, 1 means the same as patching the whole activation. Values can exceed 1. Intervals are 95% paired bootstrap intervals (1000 resamples).

Conditions (all at one layer; the subscript names the prompt each part comes from):

Features only, error only, full activation
$\hat{x}_A + \varepsilon_B$; $\hat{x}_B + \varepsilon_A$; and x_A, which gives 1 by construction.

Noise control
$\hat{x}_B + \eta$, with random noise η of the same size as ε: it erases B's error without supplying A's.

Features alone
$\hat{x}_A$ with no error at all.

Error resampling
(last token only) $\hat{x}_B$ plus the error of another pair whose prompt A has the same answer, or a different answer; and $\hat{x}_B$ plus the average training error. If the error carried the answer, same-answer resampling would beat different-answer resampling.

Positions. Every result patches all tokens from the first one where A and B differ, except pitfall 1, which patches the last token only (Figure B.1).

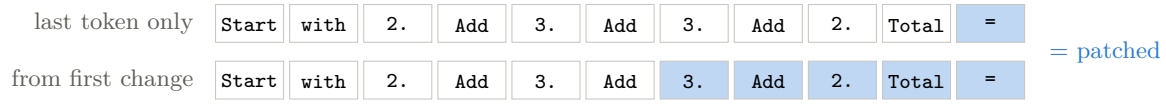


Figure B.1: Patch positions for a running-sum pair. Earlier tokens are identical in A and B, so patching from the first changed token onwards covers everything that differs; the last token alone may not yet hold the state.

Decision rules, fixed before the run. H1 is judged only at layers where patching the full activation recovers at least 0.8. A gap counts as persistent across widths if the widest SAE keeps at least 60% of the 16k gap.

B.4 Repair with error directions

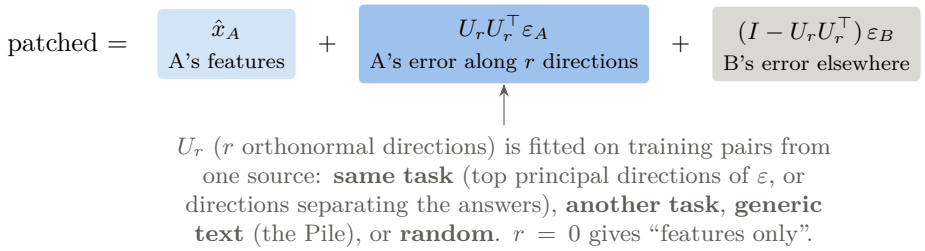


Figure B.2: Repair. The run on B receives A's features, A's error along r chosen directions, and B's own error in all other directions.

Directions are estimated on the 300 training pairs only: *top principal directions* are the directions in which the task's training errors vary most; *answer-separating directions* are the principal directions of the differences between the average errors for each answer. Other sources: the same estimate on another task (*transfer*), on generic text (the Pile, 40–100 documents of 128 tokens), or random directions (2–3 seeds). We test $r = 0, 1, 2, 4, 8, 16, 32$ (and 64 in the audit run). The **share of the gap closed** is (repair − features only) / gap.

B.5 Probes

A probe is a linear classifier trained on the training pairs to read the answer from the activation at the last token, and scored on the test pairs. We probe x, $\hat{x}$, ε and the SAE's feature activations (2048 columns), report accuracy and selectivity (accuracy minus the accuracy with shuffled labels), and compare with the same probe on the prompt's tokens alone.

B.6 Runs, compute and failures

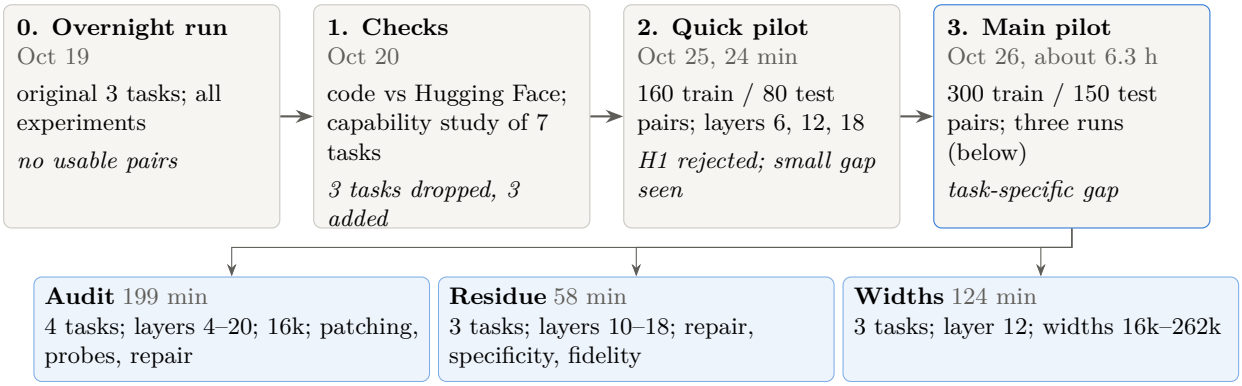


Figure B.3: The four pilot stages. The main pilot consisted of three runs on one 8 GB GPU.

Table B.3: Run inventory (all with 16k SAEs unless stated).

Run	Tasks and layers	Experiments	Time
Quick pilot	4 tasks; layers 6, 12, 18; 160/80 pairs	patching (both positions, controls); repair at 12, 18; 300 bootstrap resamples	24 min
Audit	4 tasks; layers 4, 8, 12, 16, 20	patching (both positions, controls); probes at 8–20; repair up to 64 directions	199 min
Residue	3 tasks (no variable copying); layers 10, 12, 14, 18	patching; repair with generic-text and transfer directions; fidelity	58 min
Widths	3 tasks; layer 12; widths 16k–262k	patching; repair; fidelity	124 min

Failures. In the residue run, estimating repair directions at layer 10 failed (the singular value decomposition did not converge), so there are no repair results at layer 10. In the widths run, repair for the names task at 65k ran out of GPU memory, so Table C.4 has no repair entry for that case.

C Pilot results

C.1 Features versus error, by layer

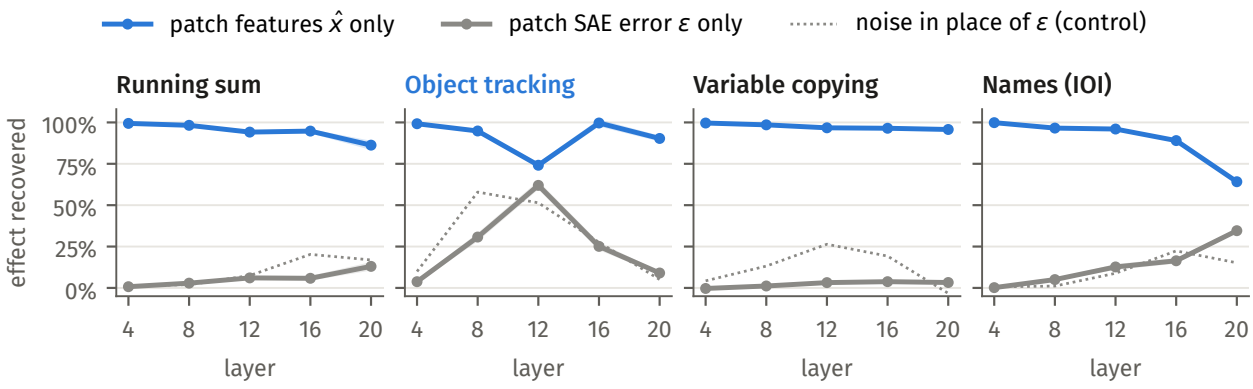


Figure C.1: The features carry most of the effect at every layer; the error alone carries little. Shaded bands are 95% intervals. Object tracking's error curve at layers 8–16 follows the noise control, so most of it comes from erasing B's error rather than from A's error.

Table C.1: Effect recovered (%) by patching the features / the error, per layer.

Task	Layer 4	Layer 8	Layer 12	Layer 16	Layer 20
Running sum	99 / 1	98 / 3	94 / 6	95 / 6	86 / 13
Object tracking	99 / 4	95 / 31	74 / 62	100 / 25	90 / 9
Variable copying	100 / 0	99 / 1	97 / 3	96 / 4	96 / 3
Names (IOI)	100 / 0	97 / 5	96 / 13	89 / 16	64 / 35

Further controls at layer 12 (effect recovered, %): the noise control gives 7, 51, 26 and 9 for the four tasks in the order of Table C.1; features alone give 102, 109, 77 and 97. For object tracking at the last token, same-answer error resampling gives 28, different-answer resampling 26 and the average error 13, against 28 for patching A's own error: the error's effect does not depend on which answer it came from, which again argues against H1.

C.2 Repair

Table C.2: Repair for every case with a gap of at least 5% (layers 8–20). Share of the gap closed (%) by one and two directions (top principal / answer-separating), by two generic-text and two random directions, and the number of directions needed to close half of the gap. Variable copying has no gap of 5% or more.

Task	Layer	Gap	1 direction	2 directions	Generic (2)	Random (2)	For half
Running sum	12	6	−2 / 9	−4 / 18	0	1	>32 / >32
	14	6	−1 / 14	−5 / 22	−1	2	>32 / >32
	16	5	−4 / 36	6 / 46	–	−1	32 / 4
	18	13	23 / 44	25 / **58**	0	1	8 / **2**
	20	14	19 / 14	23 / 43	–	0	8 / 4
Object tracking	8	5	−16 / −8	−26 / 0	–	0	>32 / >32
	12	26	**52** / 38	53 / 39	1	0	**1** / 8
	18	14	−2 / 26	18 / 36	2	1	16 / 4
	20	10	−32 / 29	−6 / 47	–	−2	16 / 4
Names (IOI)	14	10	1 / 2	1 / 2	1	0	>32 / >32
	16	11	−1 / −3	−13 / −6	–	0	>32 / >32
	18	24	6 / 5	17 / 9	0	0	32 / 32
	20	36	10 / 9	19 / 15	–	0	16 / 16

Only two cases are concentrated in the sense of the hypothesis (one or two directions close half the gap): object tracking at layer 12 and the running sum at layer 18. The other gaps need 4 to more than 32 directions, and small gaps give noisy, sometimes negative, shares. Generic-text directions were tested only in the residue run (layers 12, 14 and 18; "–" elsewhere).

C.3 Task specificity

Table C.3: Share of the gap closed (%) by two directions from each source.

Task (gap)	Source	Share closed
Object tracking, layer 12 (26%)	same task: top principal / answer-separating	53 / 39
	other task: names / running sum	2 / −1
	generic text / random	1 / 0
Running sum, layer 18 (13%)	same task: top principal / answer-separating	25 / 58
	other task: object tracking / names	1 / 0
	generic text / random	0 / 1

C.4 Width

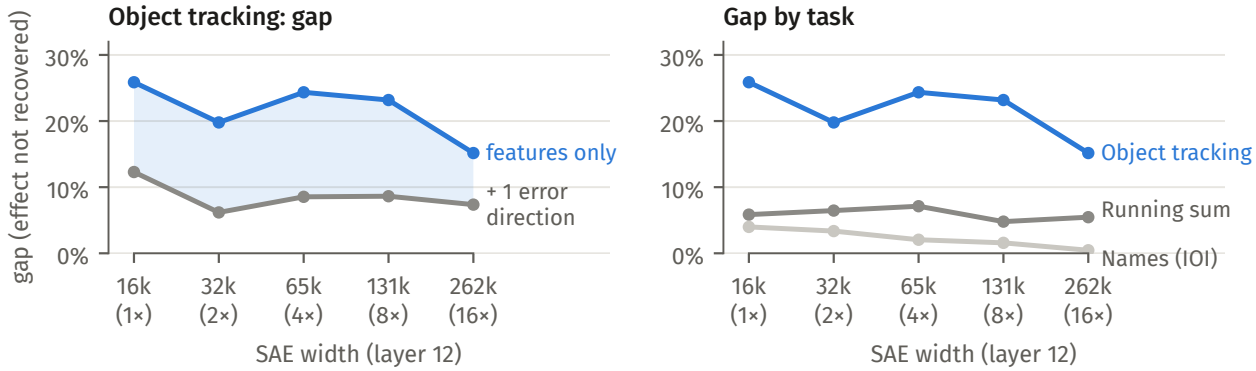


Figure C.2: Wider SAEs at layer 12. Left: the object-tracking gap with the features only and with one error direction added; the shaded band is what that direction closes. Right: the gap for three tasks.

Table C.4: Width sweep at layer 12 (%). "+1 error" and "+1 random" add one top principal or random direction; "active" is the mean number of features switched on per token.

Task	Width	Features only [95% CI]	Gap	+1 error	+1 random	Active
Object tracking	16k	74 [73, 76]	26	88	74	78
	32k	80 [79, 82]	20	94	80	75
	65k	76 [74, 77]	24	91	76	76
	131k	77 [75, 78]	23	91	77	131
	262k	85 [84, 86]	15	93	85	124
Running sum	16k	94 [93, 95]	6	94	94	80
	32k	94 [92, 95]	6	94	94	80
	65k	93 [91, 94]	7	93	93	75
	131k	95 [94, 96]	5	95	95	131
	262k	95 [93, 96]	5	94	95	129
Names (IOI)	16k	96 [95, 97]	4	96	96	89
	32k	97 [96, 98]	3	97	97	82
	65k	98 [97, 99]	2	–	–	82
	131k	98 [98, 99]	2	98	98	138
	262k	100 [99, 100]	0	100	100	139

The 262k SAE keeps 15.2% of the 25.9% object-tracking gap (59%), just under the 60% persistence rule (Appendix B.3). The 131k and 262k SAEs also switch on more features per token than the narrower ones, so part of the drop may come from sparsity rather than width.

C.5 Three pitfalls

Pitfall 1: patching only the last token. In early and middle layers the state still sits on earlier tokens, so patching only the last token recovers little even with the whole activation (Table C.5); patching from the first changed token recovers 100% at every task and layer [15].

Table C.5: Effect recovered (%) when the whole activation is patched at the last token only.

Task	Layer 4	Layer 8	Layer 12	Layer 16	Layer 20
Running sum	0	0	3	58	94
Object tracking	1	4	47	77	85
Variable copying	0	0	0	5	55
Names (IOI)	0	0	0	60	81

Pitfall 2: the prompt gives the answer away. A probe on the prompt's tokens alone scores 100%

on variable copying and names (Table C.6), so a high probe score there says nothing about the model's internal state [7]. **Pitfall 3: readable is not used.** For object tracking and names, probes read the answer far better from ε than from $\hat{x}$ (84 vs 48%; 97 vs 20%), yet patching that token recovers at most 47%: the model is not yet using the information at that position.

Table C.6: Probe accuracy at the last token, layer 12 (%), and the effect of patching that token.

Task	Probe on x	On ε	On $\hat{x}$	Prompt tokens only	Chance	Last-token patch
Running sum	57	53	42	45	25	3
Object tracking	91	84	48	33	18	47
Variable copying	98	95	39	100	11	0
Names (IOI)	100	97	20	100	11	0

C.6 Replication: quick pilot versus main pilot

Table C.7: The quick pilot (160/80 pairs) and the main pilot (300/150 pairs) agree.

Quantity	Quick pilot	Main pilot
Object tracking, layer 12: features only / error only / noise control	75 / 61 / 50	74 / 62 / 51
Object tracking, layer 12: share of gap closed by one direction	51	52
Running sum, layer 18: gap	12	13
Names, layer 18: gap; share closed by 32 directions	24; 55	24; 56

C.7 Verdict on the original hypotheses

Table C.8: What the pilot says about each original hypothesis.

Hypothesis	Verdict
H1 (error carries the state)	**Rejected** for every task: $\hat{x}$ recovers more than ε at every layer, and the error's effect does not depend on which answer it came from.
H2 (low-dimensional structure in the error)	**Not supported as stated**: the answer can be read from ε but is not used there, and for two tasks the prompt's words alone give it away.
H3 (a few directions repair)	**Supported, narrowly**: one direction closes about half of object tracking's gap at layer 12, and two close 58% of the running sum's at layer 18; the directions are task-specific. This became the project's hypothesis.

C.8 Caveats

- One model (gemma-2-2b), one SAE family (Gemma Scope) and synthetic tasks; width was tested at layer 12 only.
- Object tracking is the hardest task for the model: only 27% of generated pairs passed its filter, so its test pairs are the most heavily selected and may over-represent easier cases.
- The names control's late-layer loss (24–36% at layers 18–20) is spread over many directions: part of the late-layer loss is a general SAE weakness rather than a task-specific gap.
- Width and sparsity are confounded in the Gemma Scope family (Table C.4).
- The binding explanation is a hypothesis, and the multilingual work is a plan with no results yet.

D Design of the planned study

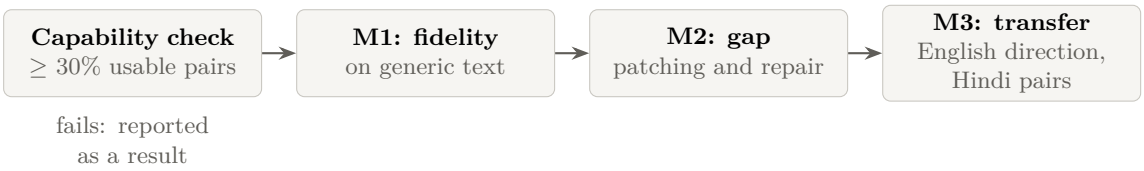


Figure D.1: Steps for each language. A language passes on only if the model can do the tasks in it.

Table D.1: Multilingual measures and criteria (95% intervals; otherwise inconclusive).

ID	Measure	Supported if	Not supported if
M1	fidelity on generic text, per language	Hindi ≥ 3 points below English	within 1 point of English
M2	gap and specificity on the tasks	Hindi gap larger, non-overlapping intervals	Hindi gap not larger
M3	English direction applied to Hindi pairs	closes ≥ half as much as Hindi's own direction	closes under a quarter as much

Languages. English (baseline); Hindi in Devanagari (the best-resourced Indian language, most likely to work); the same Hindi in Latin script without English words, and Hinglish (Latin script, mixed with English words), which together separate the effect of script from that of vocabulary; Tamil or Bengali if the model can do the tasks; French as a high-resource non-Indian control. Q2 (binding) is run in English only.

Templates. The answer must be the last token. Arithmetic templates can end with "=" as in English; templates that need a verb at the end in Hindi use a question followed by "answer:". All examples are drafts, to be checked by a native speaker and against the tokenizer:

- Object tracking (Hindi): सिक्का डिब्बे B में है। चाबी डिब्बे E में है। चाबी डिब्बे C में चली जाती है। चाबी किस डिब्बे में है? डिब्बा → C
- Names (Hinglish): `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul
- Variable copying is code-like and identical in every language: a language-neutral control.

Constraints. Prompts A and B need the same number of tokens and a one-token answer; many Hindi names split into several tokens, so names are chosen from single-token ones (the pipeline already drops and counts pairs that break this rule). **Data.** Fidelity per language uses generic text from IndicCorp v2 [3] and FineWeb2 [12].

Risks and fallbacks.

- Binding vectors may not be estimable for gemma-2-2b; Q2 then keeps only the shifting test.
- The model may not do the tasks in Tamil or Bengali; we report this and keep Hindi and Hinglish.
- Names that split into several tokens are replaced by letters or digits as answers.
- Width results at one layer may not generalise; Q1 repeats them at a second layer if time allows.

E Glossary

Table E.1: Terms used in this proposal.

Term	Meaning
Token	a word or word piece; the unit the model reads
Layer	one of gemma-2-2b's 26 processing stages
Activation x	the model's internal vector for one token at one layer (2304 numbers; also "residual stream")
Sparse autoencoder (SAE)	a tool that rewrites an activation as a few active, interpretable features
Features $\hat{x}$	the SAE's reconstruction of x from its active features
SAE error ε	$x - \hat{x}$: what the SAE does not explain ("dark matter"; informally, the leftovers)
Width	how many features an SAE can choose from (16k = 16,384, up to 262k)
Active features	the number of features switched on for one token (also L0)
Reconstruction fidelity	share of next-token performance kept when $\hat{x}$ replaces x, relative to replacing x by its average (cross-entropy loss recovered)
Prompts A and B	two prompts that differ in one step, so the right answer differs (also "clean" and "corrupt")
Activation patching	running the model on B while replacing part of its activation with A's at one layer
Logit	the model's raw score for a possible next token
Effect recovered (NLDR)	how far patching moves the model's preference from B's answer to A's, from 0 to 1
Gap	the effect the features fail to carry: one minus the effect recovered by patching the features only
Direction	a fixed vector in the 2304-dimensional activation space
Top principal directions	the directions in which a set of errors varies most
Answer-separating directions	directions that best separate the average errors for different answers
Specificity score	share of the gap closed by same-task directions minus the largest share from any other source
Probe	a simple linear classifier trained to read a value, such as the box, from an activation
Binding	how a model links an entity to its attribute, such as an object to its box
IOI	"indirect object identification", the names task [14]
Generic text	ordinary text from the Pile, used for fidelity and for generic-text directions
Hinglish	Hindi written in Latin script and mixed with English words
95% CI	bootstrap confidence interval: the range the result would likely fall in on new test pairs